

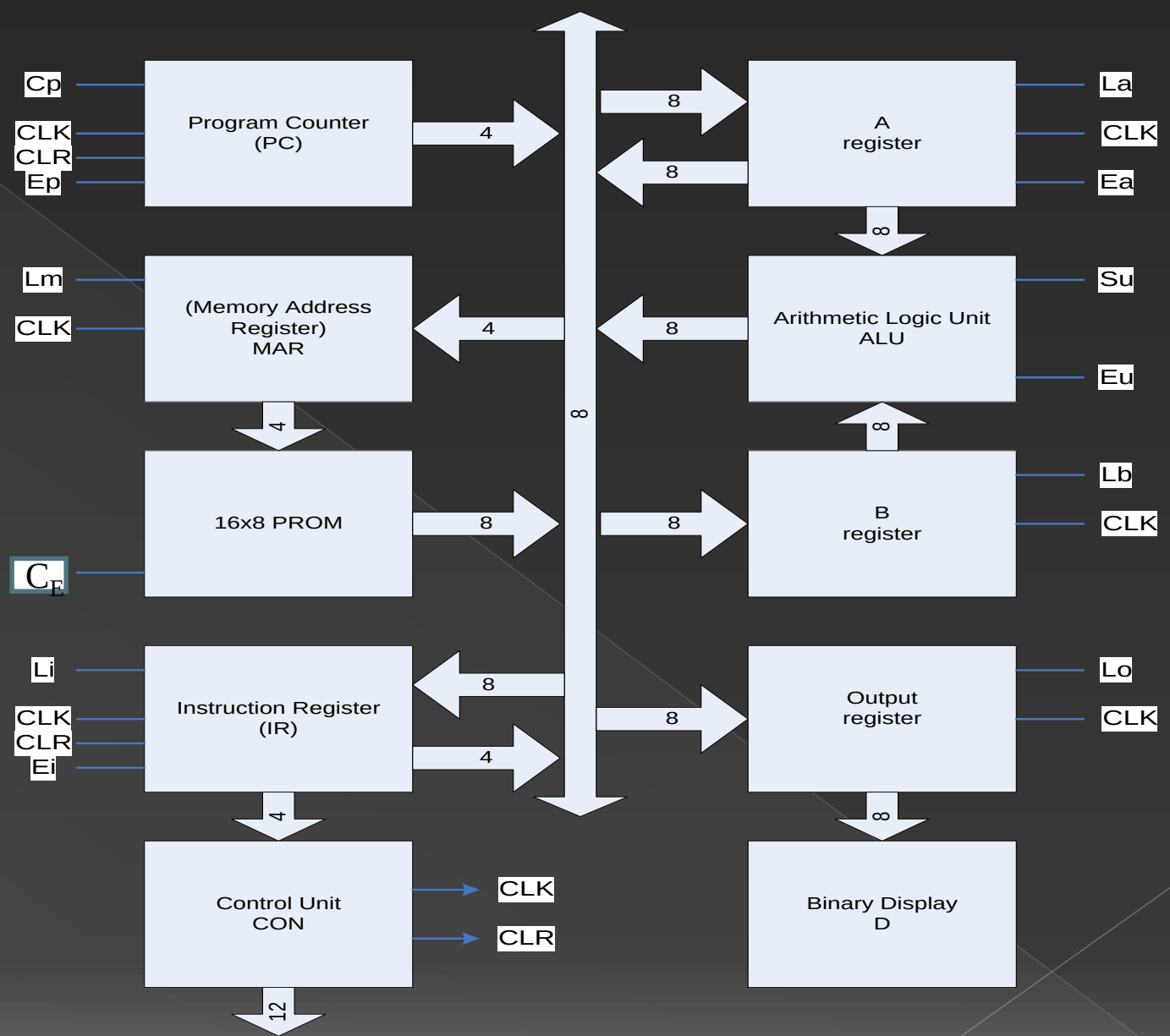
Simple As Possible -1 (SAP-1)

Computer Architecture

SAP-1 Characteristics

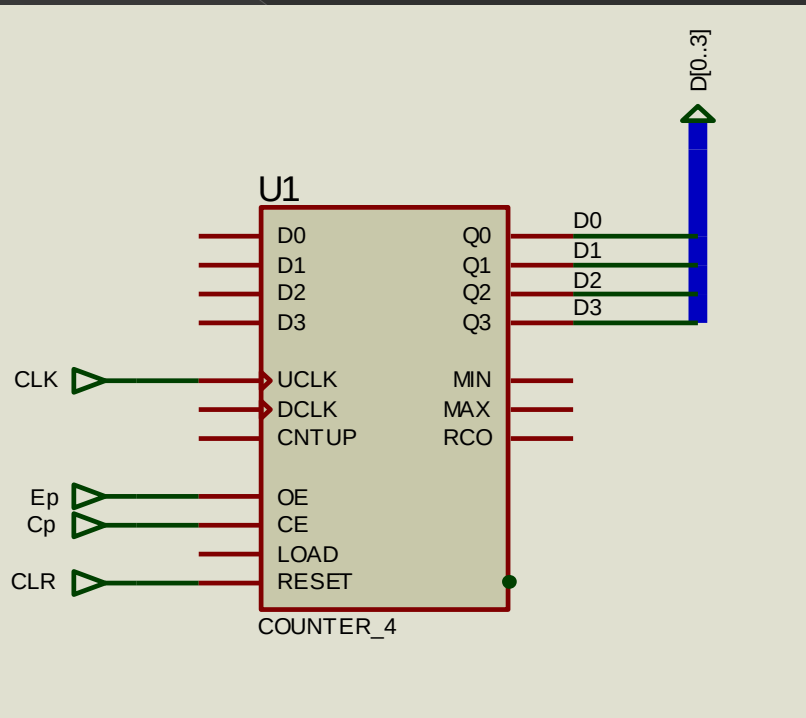
- ⦿ The 8080/8085 was the first widely used microprocessor
- ⦿ Hardwire Architecture
- ⦿ 2 (8-bit general registers)
- ⦿ 1 (8-bit output registers)
- ⦿ 4-bit ALU (additional, subtraction)
- ⦿ 4-bit instructions and 4-bits operands
 - ⦿ (cccc 0000)
 - cccc = OP CODE
 - 0000 = OPERAND
- ⦿ 16x8 address ROM for mixed program and data
- ⦿ 12 control signals

SAP-1
Architecture

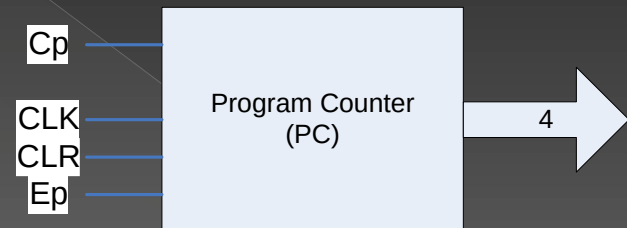


CpEpLmCpLiEiLaEa SuEuLbLo

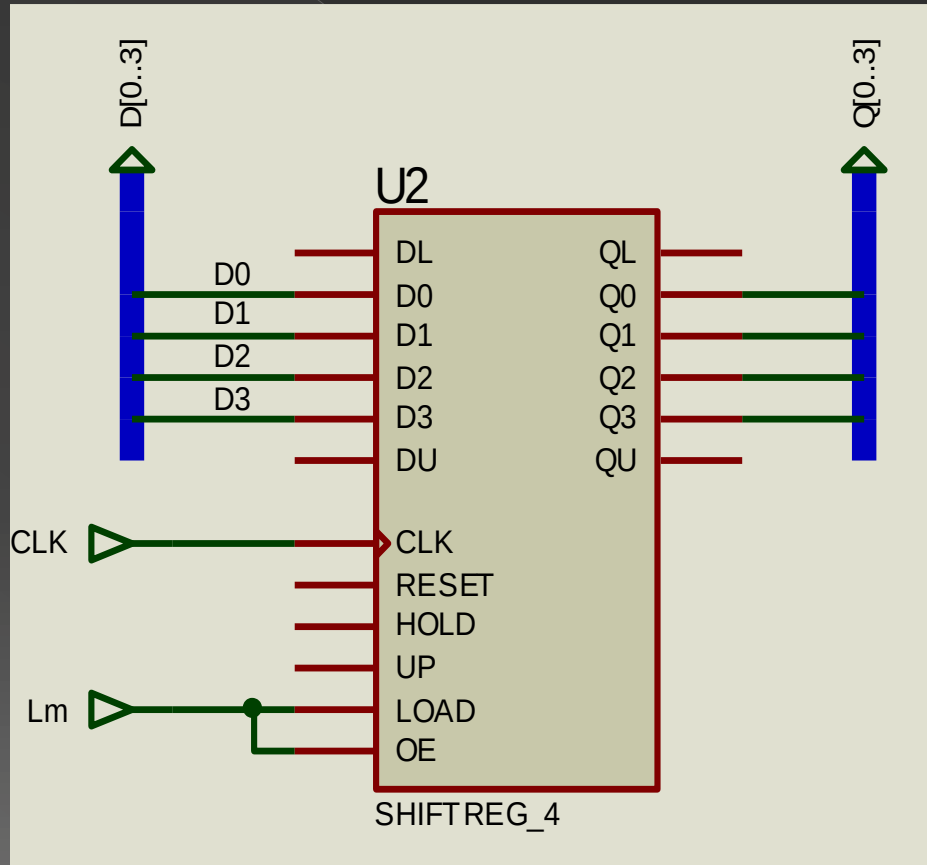
Program Counter



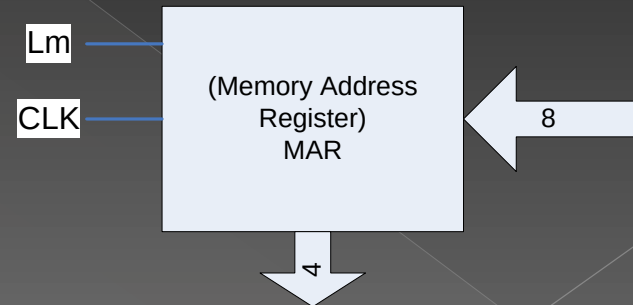
- OUTPUT: 0000-1111 (0-F)
- CLK: Clock cycle
- CLR: reset output to 0000
- Cp: $(PC) \leftarrow (PC)+1$
- Ep: output $\leftarrow (PC)$



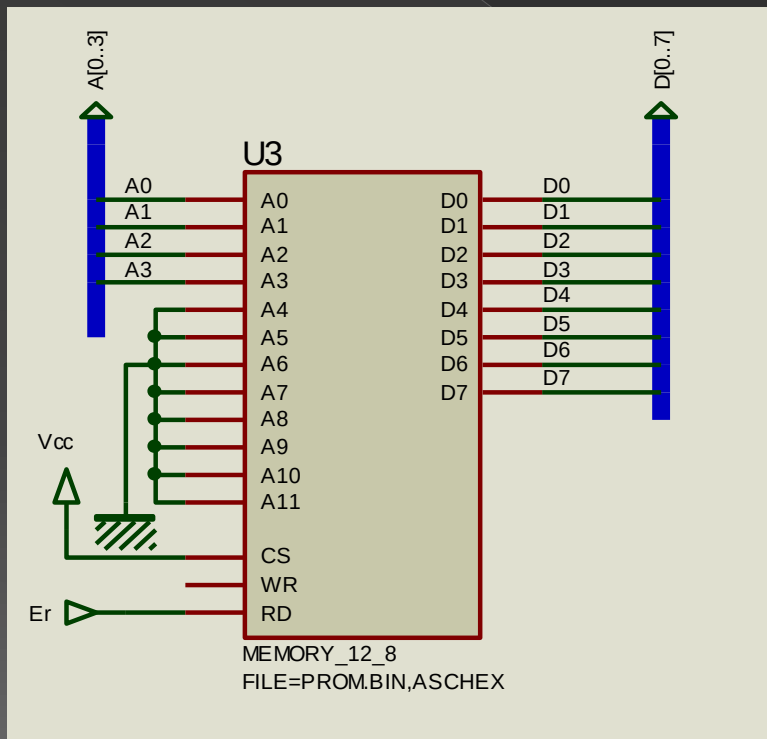
MAR



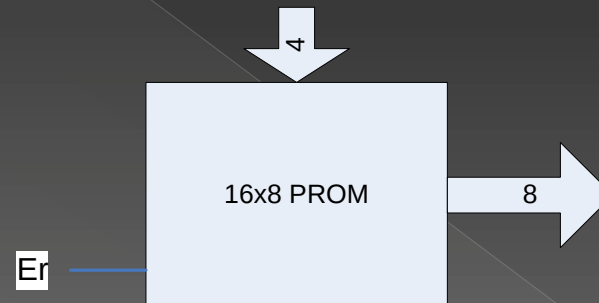
- INPUT: 8 bits (cccc 0000)
- OUTPUT: 4 bits (cccc)
- CLK: Clock cycle
- Lm:
 - (MAR) $\leftarrow$ input
 - output $\leftarrow$ HiByte(MAR)



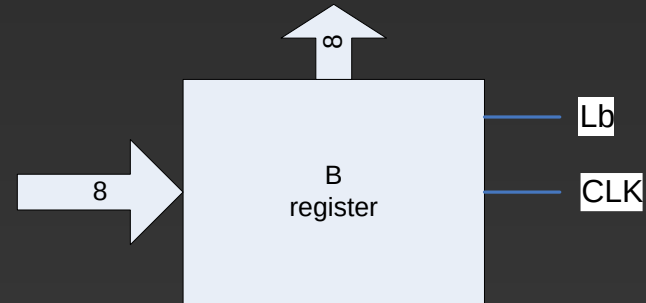
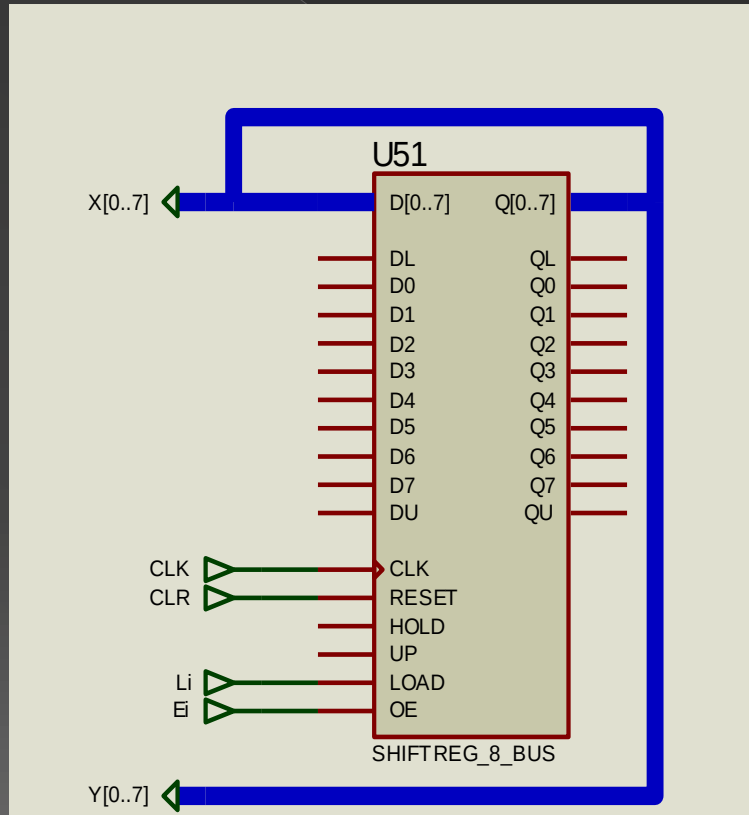
ROM (R)



- INPUT: 4 bits (cccc)
- OUTPUT: 8 bits (dddd dddd)
- Er :
 - $R \leftarrow \text{input}$
 - $\text{output} \leftarrow (R)$



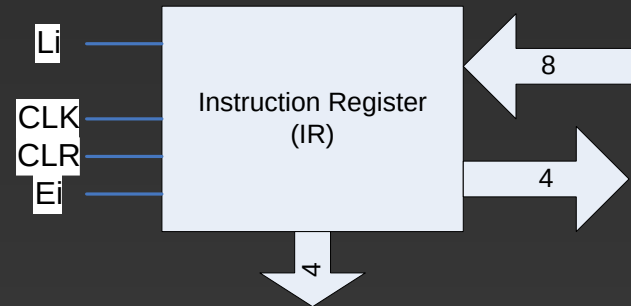
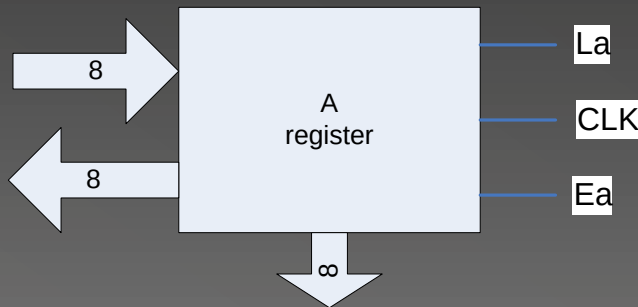
B,O Register (tri-state)



- INPUT: 8 bits (dddd dddd)
- OUTPUT: 8 bits (dddd dddd)
- CLK: Clock cycle
- Lx: Load data in
 - (A) $\leftarrow$ input
- Ex: Enable data out
 - output $\leftarrow$ (A)

I Register

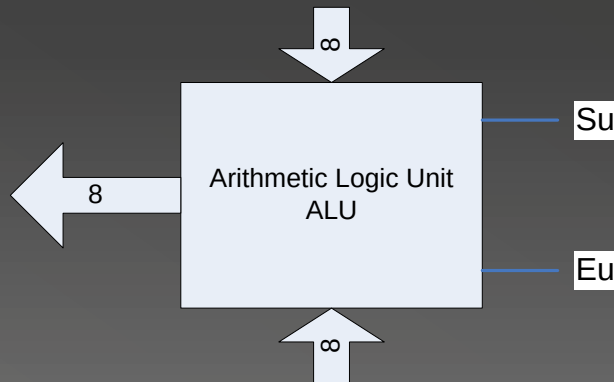
- ⊙ INPUT: 8 bits
- ⊙ OUTPUT1: 8 bits
 - ⊙ For ALU, direct output
- ⊙ OUTPUT2: 8 bits
 - ⊙ For BUS, control by Ea
- ⊙ CLK: Clock cycle
- ⊙ La: Load data in
 - ⊙ (A) $\leftarrow$ input
- ⊙ Ea: Enable data out
 - ⊙ output2 $\leftarrow$ (A)



- INPUT: 8 bits (cccc dddd)
- OUTPUT1: 4 bits (cccc)
 - For instruction decoder, direct output
- OUTPUT2: 4 bits (dddd)
 - For BUS, control by Ei
- CLK: Clock cycle
- Li: Load data in
 - (IR) $\leftarrow$ input
- Ei: Enable data out
 - output2 $\leftarrow$ (IR)

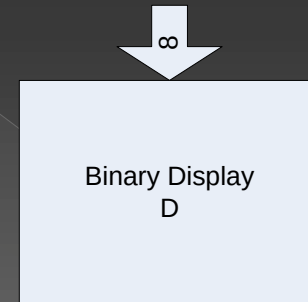
ALU

- INPUT1: 8 bits (aaaa aaaa)
- INPUT2: 8 bits (bbbb bbbb)
- OUTPUT: 8 bits (dddd dddd)
- Su: Load data in
 - 0 – additional
 - $(ALU) \leftarrow input1 + input2$
 - 1 – subtraction
 - $(ALU) \leftarrow input1 - input2$
- Eu: Enable data out
 - $output \leftarrow (ALU)$

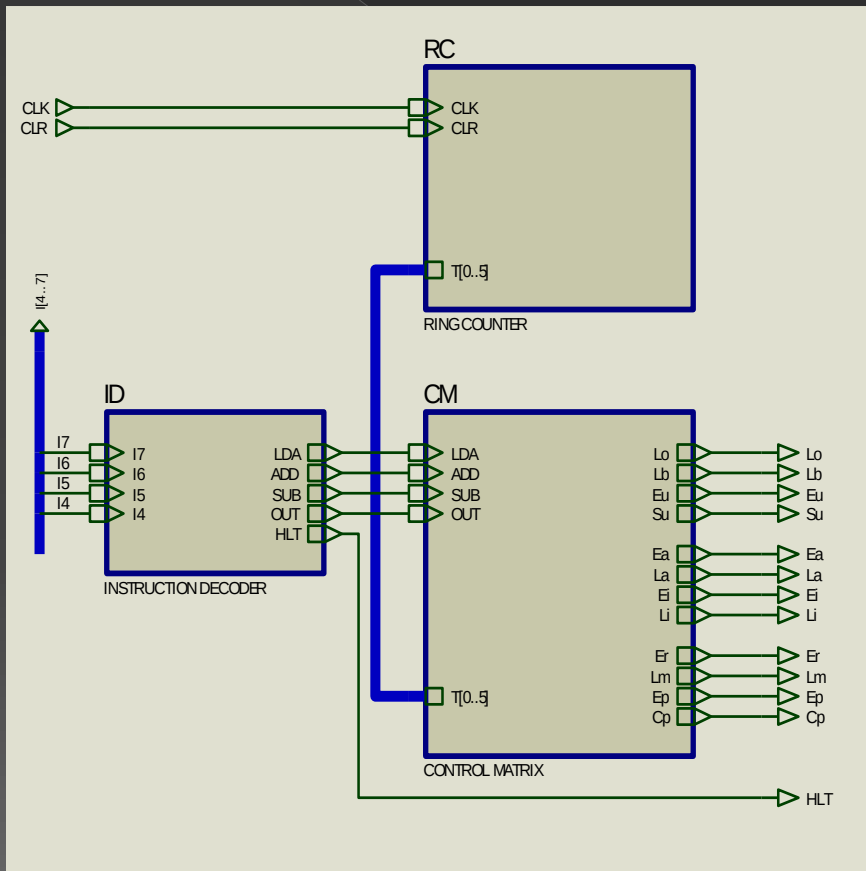


Binary Display

- INPUT: 8 bits (aaaa aaaa)



Control Unit (CON)

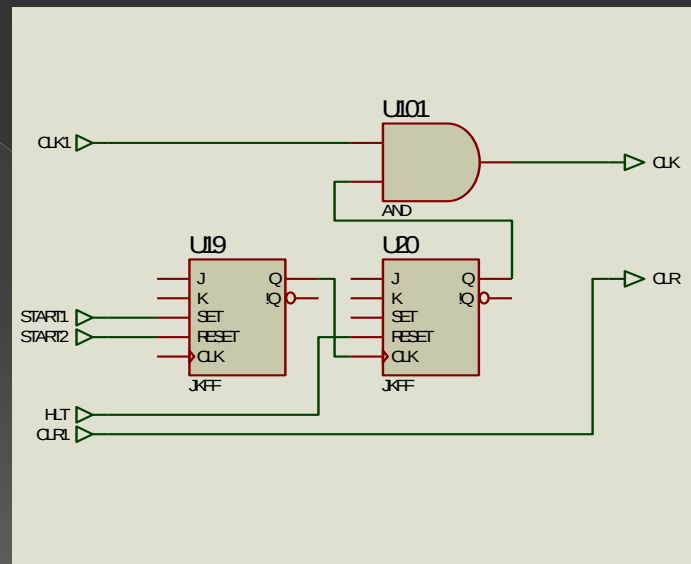
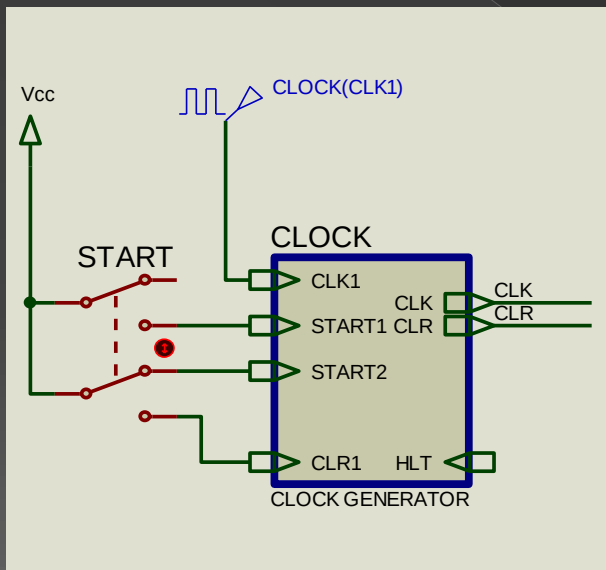


COMPONENTS

- Ring Counter
 - T0 – T5
- Instruction Decoder
 - 6 op-code signals
- Control Matrix
 - 12 control signals

CLOCK

- Press START to begin
 - Generate CLK
 - Generate CLR
- HLT Signal
 - Reset system to Initial state



OPCODE

● LDA (0000 0000)

- 0000 1001 = LDA (R9)
 - $(A) \leftarrow (R9)$

● ADD (1111 0000)

- 0001 1010 = ADD (RA)
 - $(B) \leftarrow (RA)$
 - $(A) \leftarrow (A) + (B)$

● SUB (0010 0000)

- 0010 1100 = SUB (RC)
 - $(B) \leftarrow (RC)$
 - $(A) \leftarrow (A) - (B)$

● OUT (1100 xxxx)

- 1100 xxxx = OUT
 - $(D) \leftarrow (A)$

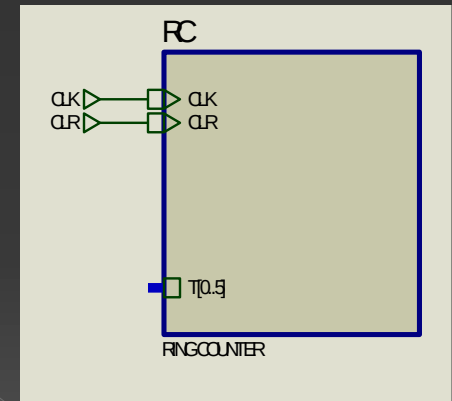
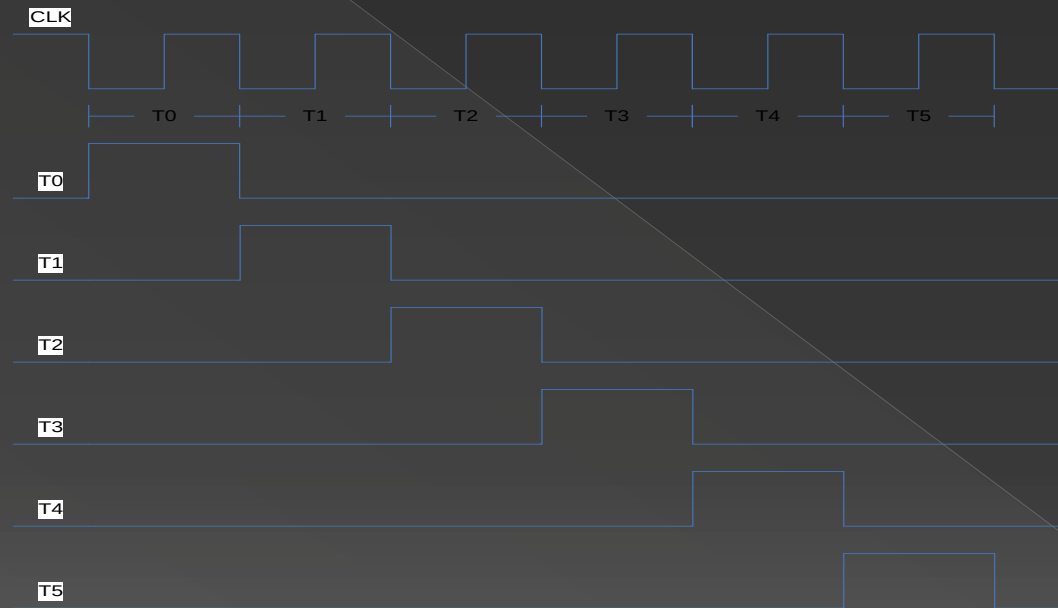
● HLT (1111 xxxx)

- 1111 xxxx = HLT

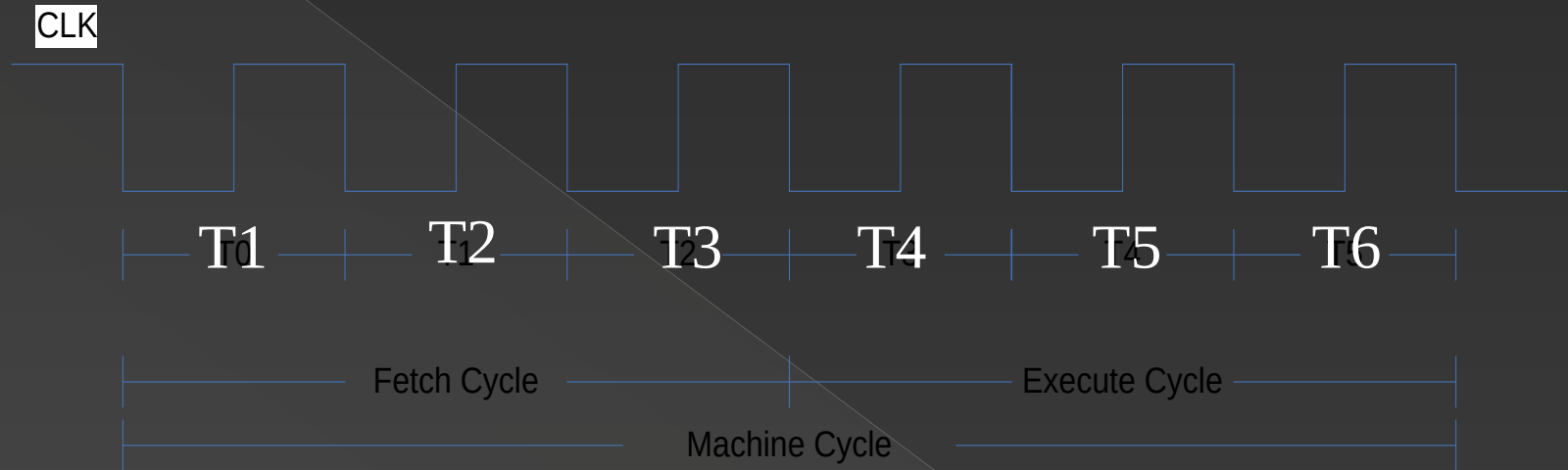
PROGRAM/DATA MEMORY

R0	-	0000 1001	(LDA R9)
R1	-	0001 1010	(ADD RA)
R2	-	0001 1011	(ADD RB)
R3	-	0001 1100	(ADD RC)
R4	-	0010 1101	(SUB RD)
R5	-	1110 xxxx	(ADD RA)
R6	-	1111 xxxx	(ADD RA)
R7	-	xxxx xxxx	
R8	-	xxxx xxxx	
R9	-	0001 0000	(16)
RA	-	0001 0100	(20)
RB	-	0001 1000	(24)
RC	-	0001 1100	(28)
RD	-	0010 0000	(32)
RE	-	xxxx xxxx	
RF	-	xxxx xxxx	

Ring Counter



Machine Cycle



- Fetch Cycle (T1-T3)
- Execute Cycle (T4-T6)

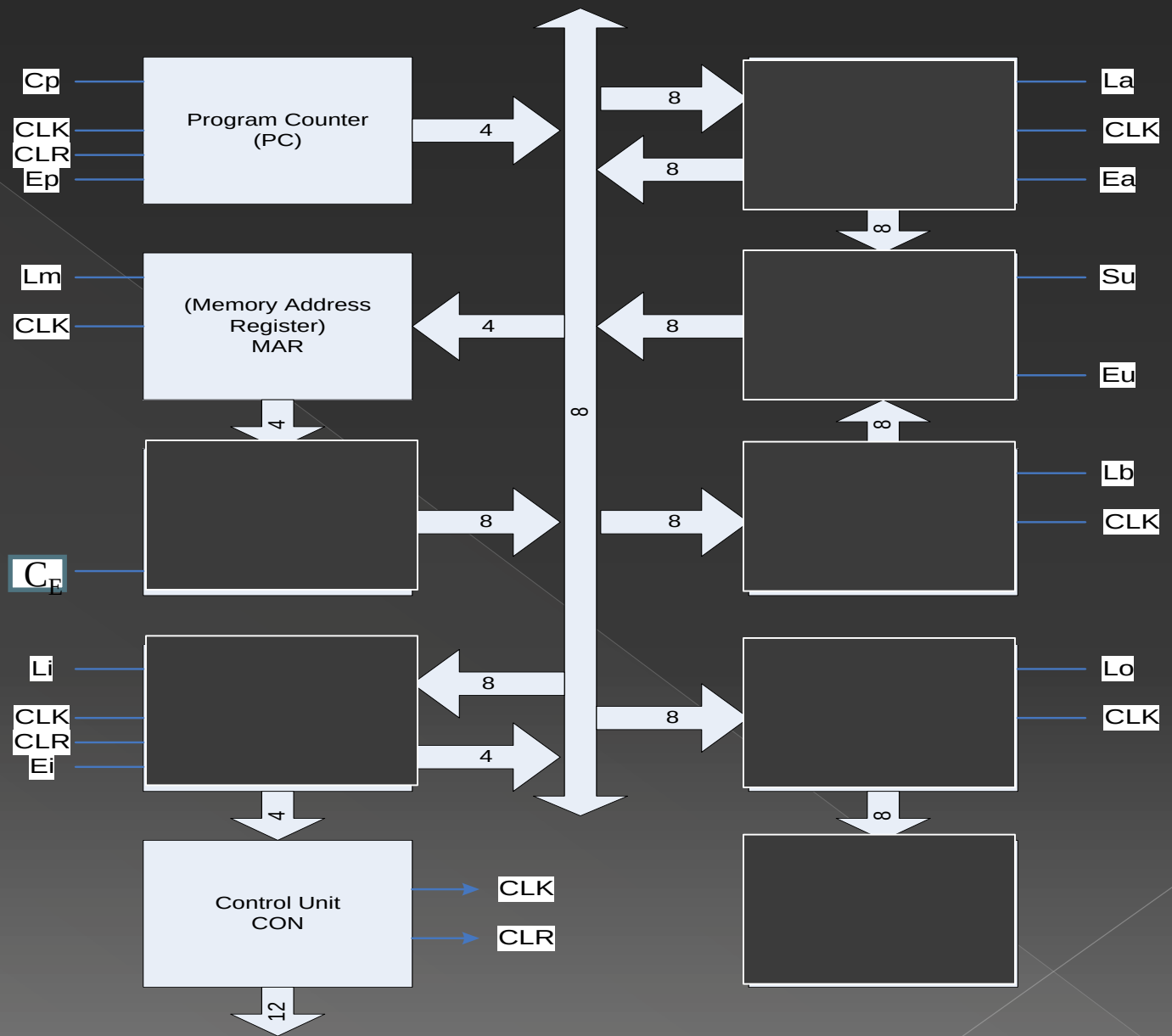
Fetch Cycle

● T1

- $EpLm \leftarrow CON$
- $(MAR) \leftarrow (PC)$

$$CON = C_P E_P \overline{L_M} \overline{CE} \quad \overline{L_1} \overline{E_1} \overline{L_A} E_A \quad S_U E_U \overline{L_B} \overline{L_O}$$

0	1	0	1	1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---



CpEpLmCfLiEiLaEa SuEuLbLo

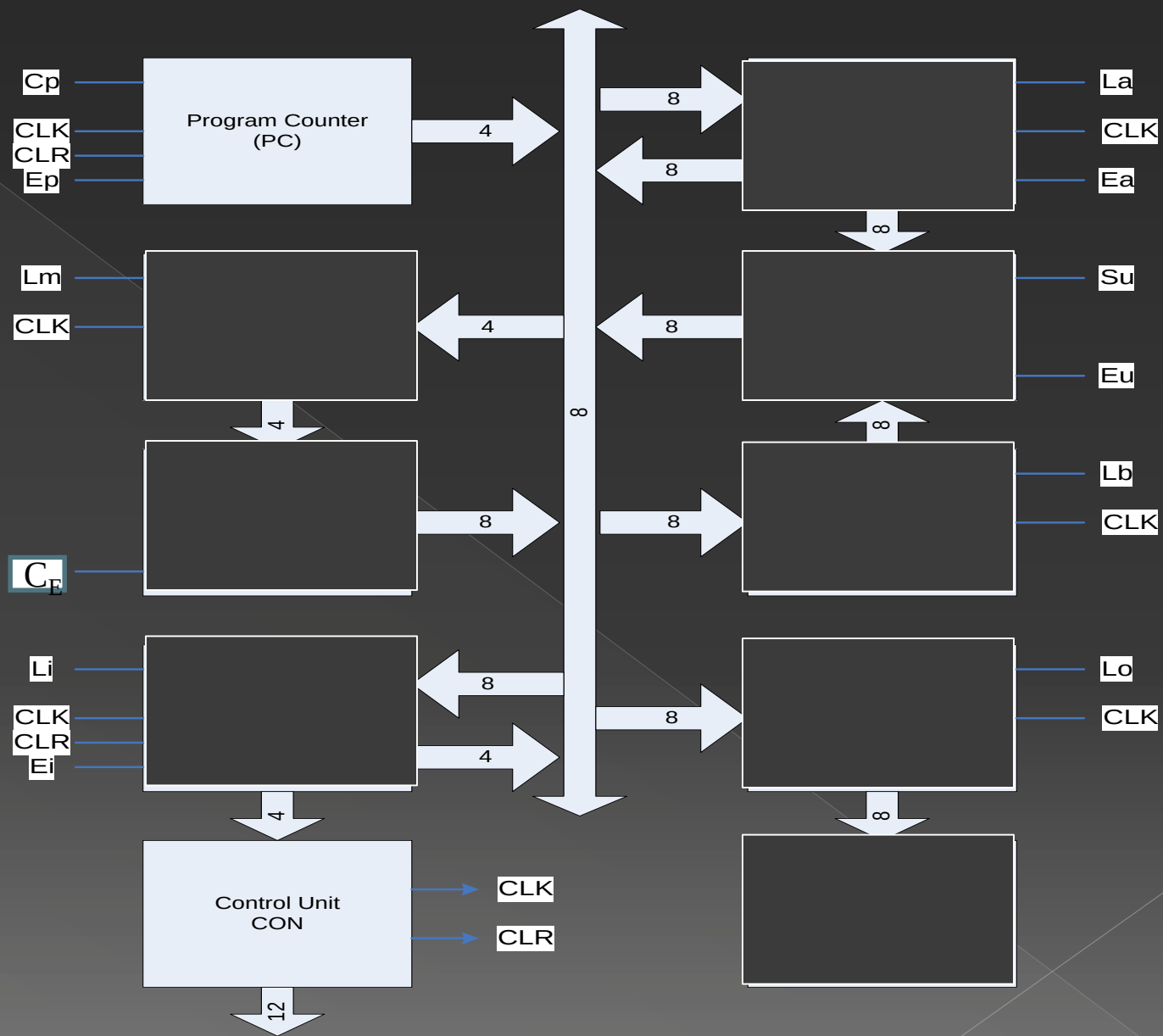
Fetch Cycle

● T2

- $ErLi \leftarrow CON$
- $(IR) \leftarrow (R \leftarrow MAR)$

$$CON = C_P E_P \overline{L_M} \overline{CE} \quad \overline{L_1} \overline{E_1} \overline{L_A} E_A \quad S_U E_U \overline{L_B} \overline{L_O}$$

1	0	1	1	1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---



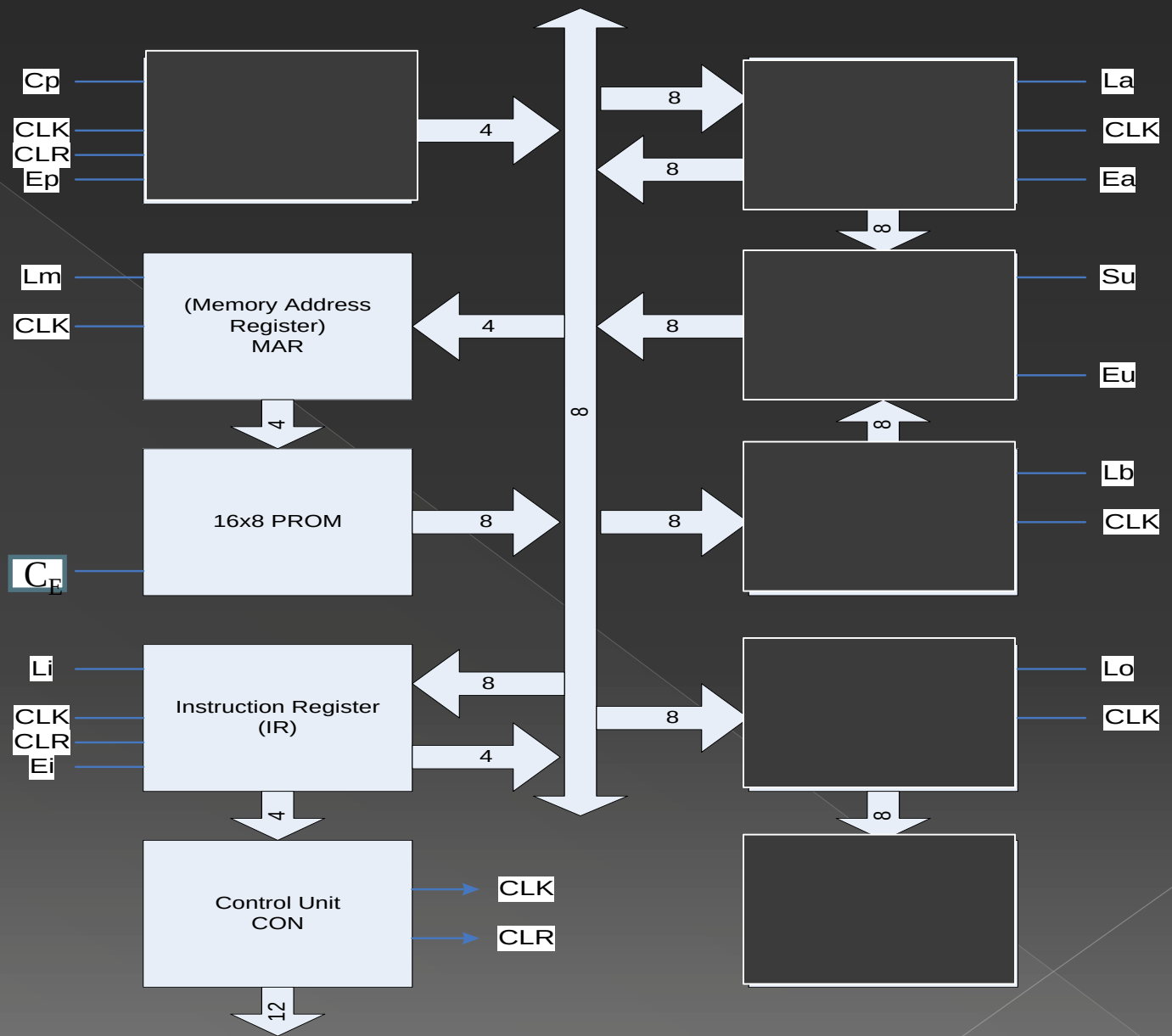
CpEpLmCpLiEiLaEa SuEuLbLo

Fetch Cycle

● T3

- $C_p \leftarrow \text{CON}$
- $(\text{PC}) \leftarrow (\text{PC})+1$

$$\begin{array}{ccccccc} \text{CON} = C_p E_p \overline{L_M} \overline{CE} & \overline{L_1} \overline{E_1} \overline{L_A} E_A & S_U E_U \overline{L_B} \overline{L_O} \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \end{array}$$



CpEpLmCp LiEiLaEa SuEuLbLo

Execute Cycle

⊙ LDA

⊙ T4

- $LmEi \leftarrow CON$
- $(MAR) \leftarrow (0000)$

⊙ T5

- $ErLa \leftarrow CON$
- $(A) \leftarrow (R0000)$

⊙ T6

- No op

⊙ ADD

⊙ T4

- $LmEi \leftarrow CON$
- $(MAR) \leftarrow (0000)$

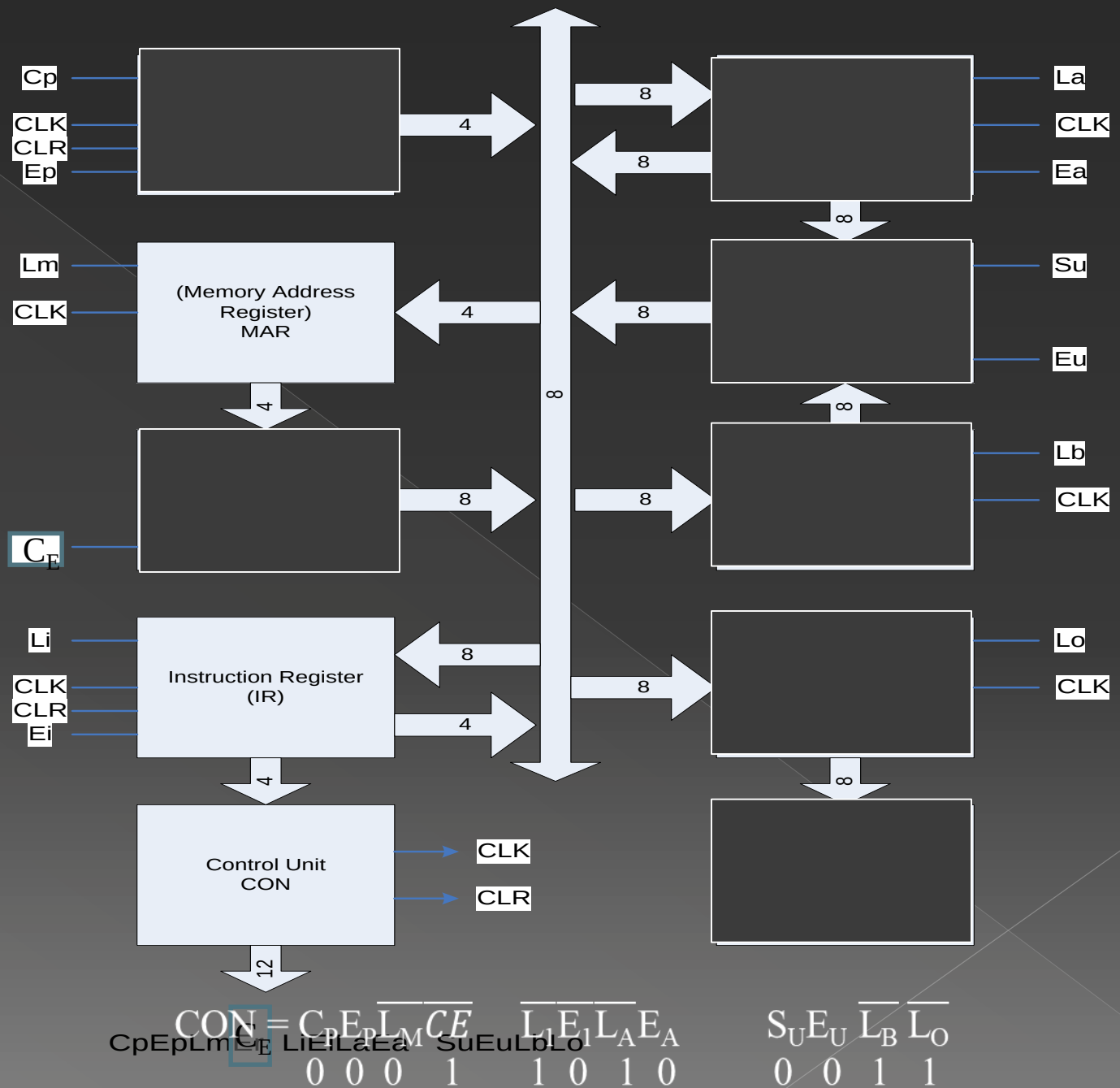
⊙ T5

- $ErLb \leftarrow CON$
- $(B) \leftarrow (R0000)$

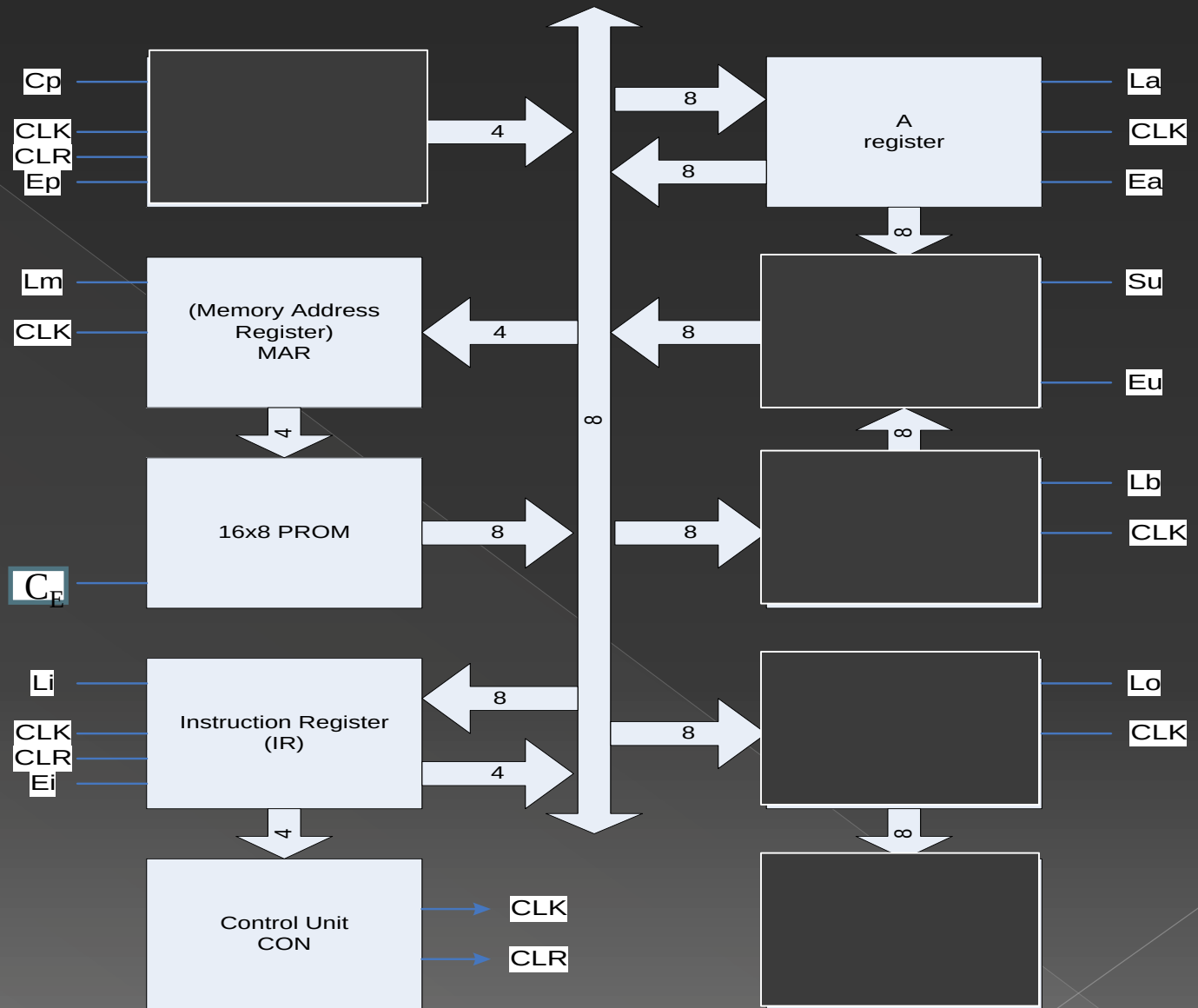
⊙ T6

- $LaEu \leftarrow CON$
- $(ALU) \leftarrow (A)+(B)$
- $(A) \leftarrow (ALU)$

T4



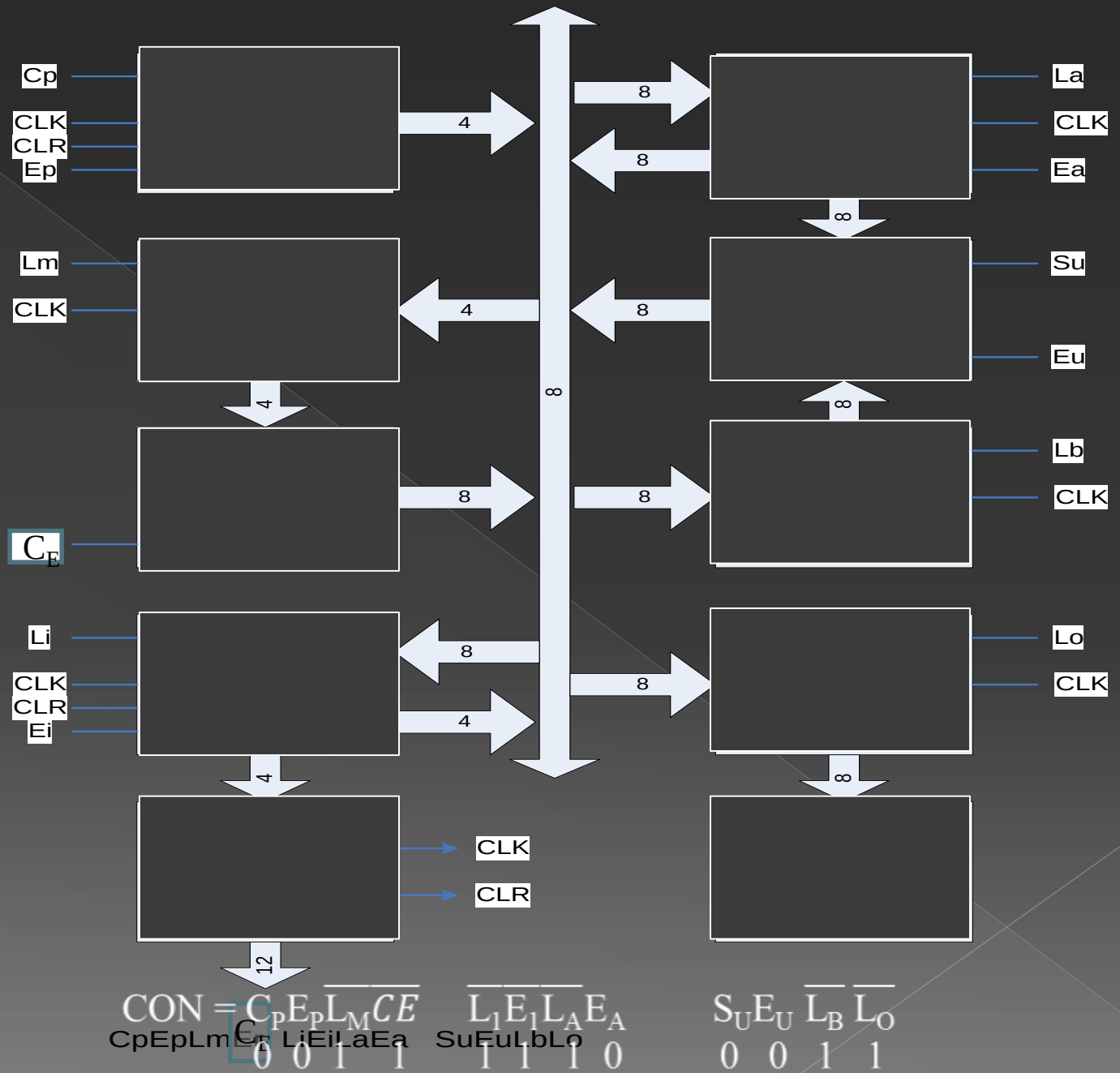
T5



$CON = C_p E_p L_m \overline{C_e}$
 $C_p E_p L_m \overline{C_e} L_i E_L \overline{L_a} E_a$
 0 0 1 0 1 1 0 0

$S_u E_u \overline{L_b} \overline{L_o}$
 0 0 1 1

T6



T1

$$\text{CON} = C_p E_p \overline{L_M} \overline{CE}$$

0 1 0 1

$$\overline{L_1} \overline{E_1} \overline{L_A} E_A$$

1 1 1 0

$$S_U E_U \overline{L_B} \overline{L_O}$$

0 0 1 1

T2

$$\text{CON} = C_p E_p \overline{L_M} \overline{CE}$$

1 0 1 1

$$\overline{L_1} \overline{E_1} \overline{L_A} E_A$$

1 1 1 0

$$S_U E_U \overline{L_B} \overline{L_O}$$

0 0 1 1

T3

$$\text{CON} = C_p E_p \overline{L_M} \overline{CE}$$

0 0 1 0

$$\overline{L_1} \overline{E_1} \overline{L_A} E_A$$

0 1 1 0

$$S_U E_U \overline{L_B} \overline{L_O}$$

0 0 1 1

T4

$$\text{CON} = C_p E_p \overline{L_M} \overline{CE}$$

0 0 0 1

$$\overline{L_1} \overline{E_1} \overline{L_A} E_A$$

1 0 1 0

$$S_U E_U \overline{L_B} \overline{L_O}$$

0 0 1 1

T5

$$\text{CON} = C_p E_p \overline{L_M} \overline{CE}$$

0 0 1 0

$$\overline{L_1} \overline{E_1} \overline{L_A} E_A$$

1 1 0 0

$$S_U E_U \overline{L_B} \overline{L_O}$$

0 0 1 1

T6

$$\text{CON} = C_p E_p \overline{L_M} \overline{CE}$$

0 0 1 1

$$\overline{L_1} \overline{E_1} \overline{L_A} E_A$$

1 1 1 0

$$S_U E_U \overline{L_B} \overline{L_O}$$

0 0 1 1

Fetch Cycle

Execute Cycle

Execute Cycle

◉ SUB

◉ T4

- $LmEi \leftarrow CON$
- $(MAR) \leftarrow (0000)$

◉ T5

- $ErLb \leftarrow CON$
- $(B) \leftarrow (R0000)$

◉ T6

- $LaSuEu \leftarrow CON$
- $(ALU) \leftarrow (A) + (B)$
- $(A) \leftarrow (ALU)$

◉ OUT

◉ T4

- $EaLo \leftarrow CON$
- $(O) \leftarrow (A)$

◉ T5

- No Op

◉ T6

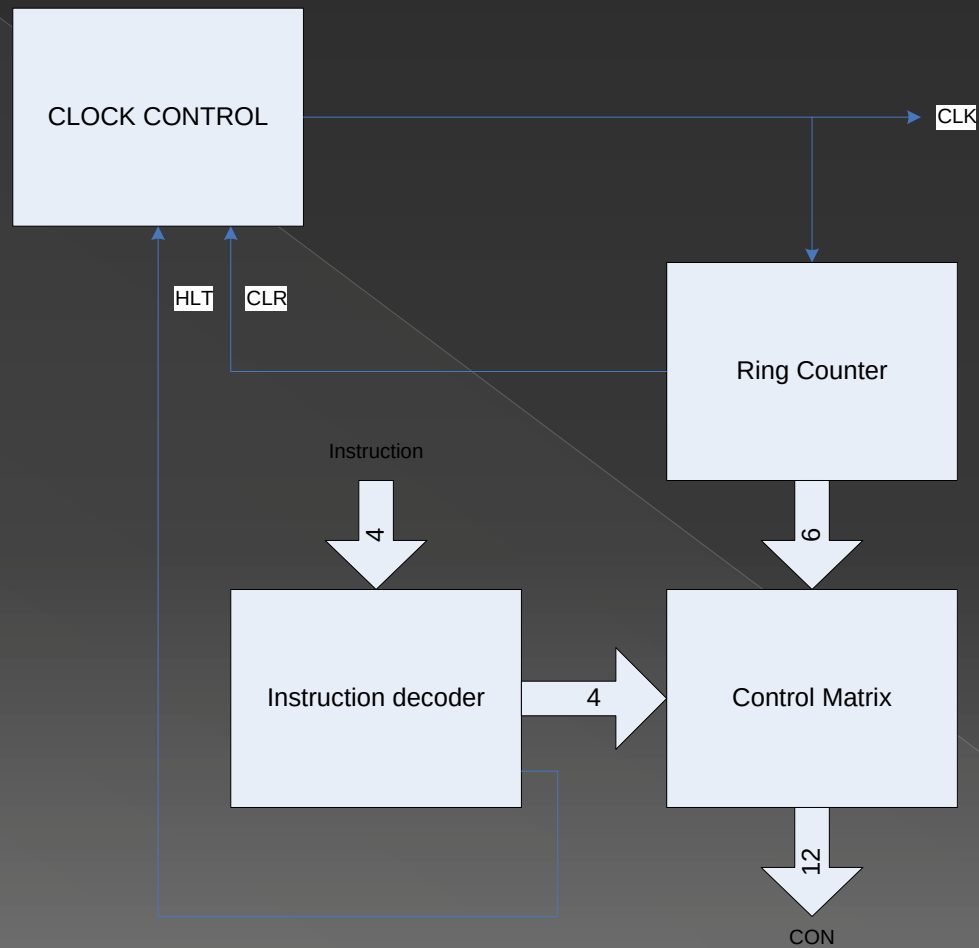
- No Op

• HLT

– T4

- HLT signal

CONTROL UNIT (CON)



Operation sample

Press start button

(PC) $\leftarrow$ 0000

Start

LDA R9

(MAR) $\leftarrow$ (PC)

fetch T0

(MAR) = 0000

R(MAR) $\leftarrow$ 0000 1001

(R) = (R0)

IR $\leftarrow$ 0000 1001

fetch T1

(PC) $\leftarrow$ (PC)+1

fetch T2

(PC) = 0001

(MAR) $\leftarrow$ IR(1ow)

exec T3

(MAR) = 1001

(A) $\leftarrow$ R(MAR)

exec T4

(A) = (R9)

NOOP

exec T5

ADD RA

(MAR) $\leftarrow$ (PC)

fetch T0

(MAR) = 0001

R(MAR) $\leftarrow$ 0001 1010

(R) = (R1)

IR $\leftarrow$ 0001 1010

fetch T1

(PC) $\leftarrow$ (PC)+1

fetch T2

(PC) = 0010

(MAR) $\leftarrow$ IR(1ow)

exec T3

(MAR) = 1010

(B) $\leftarrow$ R(MAR)

exec T4

(B) = (RA)

(A) $\leftarrow$ (A)+(B)

exec T5

(A) = 16+20

ADD RB

.
.
.

PROGRAM/DATA MEMORY

R0 - 0000 1001

(LDA R9)

R1 - 0001 1010

(ADD RA)

R2 - 0001 1011

(ADD RB)

R3 - 0001 1100

(ADD RC)

R4 - 0010 1101

(SUB RD)

R5 - 1110 xxxx

(ADD RA)

R6 - 1111 xxxx

(ADD RA)

R7 - xxxx xxxx

R8 - xxxx xxxx

R9 - 0001 0000

(16)

RA - 0001 0100

(20)

RB - 0001 1000

(24)

RC - 0001 1100

(28)

RD - 0010 0000

(32)

RE - xxxx xxxx

RF - xxxx xxxx

SAP-1 Instruction Set

Mnemonic	Operation
LDA	Load RAM data into accumulator
ADD	Add RAM data to accumulator
SUB	Subtract RAM data from accumulator
OUT	Load accumulator data into output
HLT	register Stop processing

SAP-1 Microinstructions and Macroinstructions (microprogram)

Macro	State	Contents	Active
LDA	T4	1A3h	LM, EI
	T5	2C3h	CE, LA
	T6	3E3h	none
ADD	T4	1A3h	LM, EI
	T5	2E1h	CE, LB
	T6	3C7h	LA, EU
SUB	T4	1A3h	LM, EI
	T5	2E1h	CE, LB
	T6	3CFh	LA, SU, EU
OUT	T4	2F2h	EA, LO
	T5	3E3h	none
	T6	3E3h	none

Programming SAP-1

- What does this program do?

Address	Mnemonics
0H	LDA 9H
1H	ADD AH
2H	ADD BH
3H	SUB CH
4H	OUT
5H	HLT

- The data in higher memory is

Address	Data
6H	FFH
7H	FFH
8H	FFH
9H	01H
AH	02H
BH	03H
CH	04H
DH	FFH
EH	FFH
FH	FFH

How would you program SAP-1 to solve this arithmetic problem?

- 16 + 20 + 24 - 32 =

- Assuming all numbers are in decimal form.

Seatwork

- Write an assembly language that performs this operation.

$8 + 4 - 3 + 5 - 2$

Use addresses BH to FH for the data

Convert the program into machine language version.